

Assignment 4: Self-Organizing Maps for Clustering

Title (Use of AI is strictly prohibited in case of plagiarism assignment will be rejected)

Note- in case of any points or topics out of scope of lectures feel free to learn or discuss it in doubt class

Customer Segmentation using Self-Organizing Maps (Kohonen Networks)

Problem Statement

Implement a Self-Organizing Map (SOM) to perform unsupervised clustering and dimensionality reduction on a customer dataset. You will visualize high-dimensional customer data in a 2D grid, identify customer segments, and provide actionable business insights based on the discovered patterns.

Dataset

Credit Card Customer Dataset

- **Source:** [Kaggle - Credit Card Dataset for Clustering](#)
- **Alternative:** [UCI - Wholesale Customers](#)
- **Description:** 8,950 customers with 18 behavioral features (credit limit, purchases, payments, tenure, etc.)

Detailed Instructions

Part 1: Data Preparation & EDA (15%)

1. Load the customer dataset and handle missing values
2. Perform exploratory data analysis:
 - Feature distributions (histograms, box plots)
 - Correlation matrix heatmap
 - Identify outliers using IQR method
3. Feature engineering (if needed):
 - Create ratio features (e.g., purchases/credit_limit)
 - Derive categorical segments
4. Data preprocessing:
 - Normalize features to [0, 1] using Min-Max scaling
 - Standardize using z-score normalization (compare both)
5. Dimensionality check: Perform PCA to visualize data in 2D

Part 2: SOM Implementation (40%)

Task 2.1: Build SOM from Scratch or Using Library

- **Option A:** Implement SOM using MiniSom library (pip install minisom)
- **Option B:** Implement from scratch using NumPy

Task 2.2: SOM Configuration

1. **Grid Configuration:**
 - Map size: 10×10 or 15×15 grid
 - Topology: Rectangular or Hexagonal
 - Initialization: Random or PCA-based
2. **Training Parameters:**
 - Learning rate: Start at 0.5, decay to 0.01
 - Neighborhood function: Gaussian
 - Neighborhood radius: Start at 3, decay to 1
 - Training iterations: 10,000-50,000
3. **Distance Metric:** Euclidean distance

Task 2.3: Training Procedure

1. Initialize weight vectors randomly or using PCA
2. For each training iteration:
 - Select random input vector
 - Find Best Matching Unit (BMU)
 - Update BMU and neighbors using learning rate and neighborhood function
3. Track quantization error over iterations
4. Save trained SOM weights

Part 3: Visualization & Analysis (30%)

Task 3.1: SOM Visualizations

1. **U-Matrix (Unified Distance Matrix):**
 - Visualize distances between neighboring neurons
 - Identify cluster boundaries (high distances)
2. **Component Planes:**
 - Create heatmaps for each feature

- Show how each feature is distributed across the map

3. **Hit Map:**

- Show frequency of activations for each neuron
- Identify popular clusters

4. **Sample Mapping:**

- Map all customers to their BMUs
- Color-code by predicted cluster

Task 3.2: Cluster Identification

1. Apply hierarchical clustering or K-means on SOM weights
2. Identify 3-5 distinct customer segments
3. Color the SOM map by cluster membership

Task 3.3: Segment Profiling

1. For each discovered cluster:
 - Calculate mean feature values
 - Identify distinguishing characteristics
 - Assign meaningful labels (e.g., "High-value spenders", "Dormant users")
2. Create radar charts comparing clusters
3. Statistical comparison (ANOVA) of features across clusters

Part 4: Business Insights (15%)

1. Interpret each customer segment in business terms
2. Provide marketing recommendations for each segment
3. Identify at-risk customers (potential churners)
4. Suggest cross-selling opportunities
5. Create a one-page executive summary with key findings

Expected Outputs

1. **Code Deliverables**

- Data preprocessing script
- SOM training script (from scratch or using library)
- Visualization notebook
- Cluster analysis script

2. **Model Files**

- Trained SOM weights (NumPy array or pickle)
- Cluster assignments for all customers
- Preprocessed data with cluster labels

3. **Visualizations** (minimum 10)

- Feature correlation heatmap
- PCA scatter plot (2D projection)
- U-Matrix visualization
- Component planes (one for each feature)
- Hit map with frequency counts
- Cluster-labeled SOM map
- Radar charts for cluster profiles
- Box plots comparing feature distributions across clusters
- Quantization error convergence plot
- 3D visualization of top 3 principal components colored by cluster

4. **Cluster Profiles Document**

- Detailed description of each segment
- Statistical summary (mean, std) of features per cluster
- Percentage of customers in each cluster
- Business interpretation and recommendations

5. **Final Report** (4-5 pages)

- Introduction to SOM and its advantages for customer segmentation
- Methodology and parameters chosen
- Cluster analysis and interpretation
- Business insights and actionable recommendations
- Comparison with K-means clustering
- Limitations and future work

Findings/Observations to Document

1. How does SOM preserve topological relationships in the data?
2. What is the optimal map size for this dataset?
3. How does the choice of learning rate and neighborhood function affect convergence?
4. Which features contribute most to customer segmentation?
5. How do SOM-based clusters compare to K-means clusters?
6. What interesting patterns or outliers did the SOM reveal?
7. How can businesses use these segments for targeted marketing?

Evaluation Rubric (100 points)

Criteria	Points	Description
Code Implementation	30	Correct SOM implementation, proper training procedure, functional code
Visualizations	25	U-Matrix, component planes, hit map, cluster profiles - clear and informative
Cluster Analysis	20	Meaningful segments identified, statistical validation, profiling
Business Insights	15	Actionable recommendations, clear interpretation, executive summary
Report Quality	10	Well-structured, clear explanations, professional presentation

Bonus Points (10)

- Implement SOM from scratch without libraries (+5)
- Compare rectangular vs. hexagonal topology (+2)
- Interactive dashboard (Plotly/Dash) for cluster exploration (+3)
